

ANTICIPATED TOUGH QUESTIONS

Bidaw 예상 까다로운 질문 30선

전문가 청중 방어용

Bidaw — FAST '26 paper seminar 보조자료

Bidaw: Enhancing Key-Value Caching for Interactive LLM Serving via Bidirectional Computation-
Storage Awareness

Hu et al., FAST '26

전문가 청중이 던질 만한 질문 + 모범 답변. 발표 전 한 번 입으로 답해보고, 답이 막히면 학습 노트로 돌아갈 것.

분류 - **C** = 개념/정의 (Concept) - **D** = 설계 결정 (Design) - **E** = 평가 / 일반화 (Evaluation) - **L** = 한계 / 비판 (Limitation)

A. 문제 정의와 워크로드

Q1. [C] "Interactive LLM serving"이라고 별도 분류하는 이유가 무엇인가? ShareGPT나 Mooncake도 멀티턴이 있는데?

A. 두 trace 모두 **interactive**에는 부적합하다. ShareGPT는 평균 5.7 라운드로 **너무 짧고**, Mooncake는 평균 query 12k 토큰으로 **길지만 사용자 turn 사이의 시간 정보가 없다**. 본 논문은 (1) 평균 22.4 라운드, (2) 사용자 단위 timestamp, (3) 분 단위 사용자 사고 시간을 모두 갖춘 trace를 **처음으로** 정의해 사용. 이 trace에서만 보이는 약한 temporal locality와 답변-길이 신호를 분석한 게 contribution의 일부.

Q2. [E] 이 trace는 China Telecom에서 받은 것이라고 하는데 **공개되는가?** 재현 가능성은?

A. 논문 footnote에 GitHub 링크가 있다 ([ShipengHu-777/Interactive-conversation-workload](https://github.com/ShipengHu-777/Interactive-conversation-workload)). Trace 자체는 공개. Reproducibility는 SSD 환경(1.5 GB/s)에 의존하고, A800 GPU도 일반 연구자가 접근하기 어려울 수 있음 — 이걸 본 논문의 reproducibility 약점.

Q3. [D] **Weighted reuse distance**를 정의한 게 핵심인데, 왜 그냥 **거리 (몇 번째 access인가)**가 아닌가?

A. Cache 압력은 **byte 단위**이지 **횟수 단위**가 아니다. 같은 횟수 거리라도 그 사이 큰 KV가 들어오면 perf layer가 더 빨리 압박된다. 논문이 보여주는 "80% access > 200 GB"는 **byte 단위**로 표현해야 perf layer 크기와 직접 비교 가능. 횟수 거리로는 perf layer 크기와 직접 매핑이 안 된다.

Q4. [L] 왜 한 사용자 안 KV 재사용만 다루고 **cross-user**는 안 다루는가?

A. 명시적으로 범위 밖이라고 § 6에서 인정. MeanCache(arXiv '24) 류는 prompt 의미가 비슷하면 다른 사용자끼리 KV 재사용을 시도하는 직교한 방향. Bidaw + MeanCache는 결합 가능하다. 본 논문 contribution을 **single-user multi-round**에 한정해 분석을 깊게 한 trade-off.

B. Mechanism 1 — I/O-aware Scheduling

Q5. [D] 왜 단순 prefetch / overlap으로 안 풀리는가? CUDA stream으로 KV load와 compute를 겹치면 되지 않나?

A. 한 iteration GPU compute는 수십 ms, capacity-layer KV load는 수백 ms. 한 iteration overlap은 수십 ms만 가린다. 게다가 GPU는 "다음 토큰" 단위로 stall하므로 다음 layer 단위 prefetch가 의미가 적다. 결국 큐 자체를 재구성해서 큰 I/O 요청이 큐 머리를 차지하지 못하게 해야 한다.

Q6. [C] disk-HRRN이 원래 HRRN과 다른 점이 무엇인가?

A. HRRN의 service time을 KV size로 대체했다. 가정: capacity layer I/O 시간 $\approx$ KV size / SSD bandwidth. 즉 size가 곧 service time의 proxy. 그 외엔 동일 — $R = 1 + \text{waiting}/\text{service}$ 라는 골격은 그대로.

Q7. [D] 큰 KV가 결국 starvation 되는 코너 케이스는?

A. 식이 $R = 1 + \text{waiting}/\text{size}$ 이므로, waiting이 충분히 늘면 R이 작은-KV 신규 요청보다 커진다. 따라서 finite waiting time 후 promote 보장. 논문이 명시하지는 않지만 식의 monotonicity로부터 도출됨. Tail latency 관점에서도 P99에서 -47% 감소했다는 게 starvation이 실제 현상에서 발생하지 않음을 시사.

Q8. [D] Promote된 요청이 ready queue에 끼어들 때 위치는 어떻게 정하는가?

A. 원래 도착 시각을 기준으로 ready queue 안에서의 position 결정. Promotion 완료 시점이 아니다. 이게 없으면 promote 늦은 요청은 끝까지 밀리고 P99 폭주. 이건 분명히 발표 시 짚어야 할 디테일.

Q9. [E] Scheduling 효과가 $1.58\times$ — 이 숫자는 어디서 나오는가?

A. Figure 21 (ablation). vanilla(vLLM + 2-tier baseline)에 I/O-aware scheduling만 추가한 버전과 비교. 동일 latency 기준 throughput으로 측정.

Q10. [L] 만약 SSD가 아니라 NVMe-PCIe5 (10+ GB/s)로 바뀌면? Scheduling이 의미 없어지지 않나?

A. 논문이 직접 답했다 — 5 GB/s SSD를 시뮬레이션해도 FlashGen 27.81 $\rightarrow$ 30.35 user/min, Bidaw는 여전히 큰 격차 유지. Bandwidth가 늘어도 KV size 분산에서 오는 head-of-line blocking은 그대로이기 때문. 다만 격차가 좁혀지는 건 사실.

C. Mechanism 2 — Eviction

Q11. [C] Spearman $\rho = 0.94-0.98$ 이 답변 길이 vs reuse distance인가, 아니면 답변 길이 vs reuse distance의 하한인가?

A. 후자다. 답변이 길수록 reuse distance가 최소 얼마 이상이 된다는 하한 관계. 평균이나 상한이 아니다. 그래서 § 3.3.2에서 "더 작은 bucket의 확률을 0으로 truncate"라는 사용 방식이 자연스럽다 — 하한 정보로는 작은 쪽을 잘라내는 것만 가능.

Q12. [D] Belady's algorithm은 미래를 알아야 하는데 왜 ghost cache에서 가능한가?

A. Ghost cache는 이미 지나간 trace만 보고 있다. 그 trace에 대해서는 미래(=현재)를 이미 알고 있으므로 Belady가 사후적으로 시뮬레이션 가능. 미래의 진짜 미래를 예측하는 게 아니라, 과거 데이터로 "Optimal이라면 hit률이 얼마였을까"를 측정. 그 측정값을 다음 access의 hit 확률 추정치로 사용.

Q13. [D] Ghost cache 자체의 메모리 비용은?

A. Ghost cache는 데이터가 없고 metadata만 (KV의 ID, size 등). 한 metadata entry가 수십 byte 수준이므로 수백만 access를 추적해도 MB 단위 메모리. Eviction 결정 단계의 추가 latency 0.35 ms — Belady 시뮬레이션은 백그라운드에서 처리되므로 결정 시점에는 lookup만.

Q14. [D] Promising bucket을 몇 개(m)로 자르는가? 어떻게 정했는가?

A. 논문이 정확한 m 값을 명시하지 않지만 "fine-grained"라는 표현으로 봐서 수십 개 단위. Bucket 크기를 작게 하면 hit rate 추정 정확도 ↑, 메모리/계산 비용 ↑. Empirical하게 결정한 것으로 보이며, 본 논문이 hyperparameter 민감도 분석을 생략한 부분 — reviewer가 지적할 수 있는 약점.

Q15. [D] Equation 2의 $\text{prob_extreme} \cdot 0.0$ 은 자명한데 왜 식에 명시했는가?

A. 가독성과 정의 완결성. 세 영역의 분포가 합 = 1임을 강조하고, extreme region을 명시적으로 0 hit로 처리한다는 설계 결정을 분명히 하는 의미. 또한 truncation 후 정규화에서 prob_extreme 항이 자동으로 0이 되지 않는 케이스를 다루기 위해.

Q16. [E] Eviction miss rate -57.6% (vs queue-enhanced) — 이 차이가 정말 답변 길이 신호 덕분인가, 아니면 bucketing 덕분인가?

A. 둘 다. Bucketing 만으로는 distribution 추정만 가능 (compute 정보 없음). 답변 길이 신호가 분포의 하한을 truncate하면서 정확도를 올린다. 둘을 분리한 ablation은 논문에 없음 — 이걸 reviewer가 추가로 요구할 수 있는 분석.

Q17. [L] ShareGPT에서 Bidaw 효과가 약해지는 이유 (Figure 17)?

A. ShareGPT는 사용자 timestamp가 없어 Poisson 시뮬로 시간 보간. 이 시뮬은 답변 길이와 사용자 사고 시간의 상관관계를 깬다. 따라서 본 논문이 사용한 신호 (compute → storage)의 정확도가 떨어져 eviction 효과가 약화. Scheduling은 영향 적어 throughput +1.40× 유지.

Q18. [C] "다음 access의 hit potential"을 계산하는데, 같은 사용자의 같은 KV가 여러 번 다시 access된다는 가정은 맞는가?

A. 맞다. Multi-round 구조 때문. 라운드 N의 응답을 만들려면 라운드 0, 1, ..., N-1의 모든 KV를 읽어야 한다. 따라서 KV1은 라운드 1, 2, 3, ...에서 모두 access. 사용자의 latest answer 길이를 보고 "next access"의 reuse distance를 추정한다는 의미.

D. Mechanism 3 — Storage-Efficient Tensor

Q19. [C] "Tensor 6"이 무엇인지 한 문장으로?

A. Decoder layer의 forward 흐름 중 FFN 직전, LayerNorm 통과 후의 정규화된 activation. KV tensor는 그 다음 단계의 Q,K,V projection 결과. Tensor 6에서 한 번 더 GPU step을 거치면 KV tensor를 만들 수 있다.

Q20. [D] Cost efficiency = saved compute / required space — 두 항을 어떻게 측정했는가?

A. Saved compute = 만약 그 tensor를 재계산했어야 했다면 들었을 GFLOPs. Required space = 그 tensor를 저장하는 데 필요한 MB. OPT-13B, 2048 토큰 history에서 Tensor 6 = 51 GFLOPs/MB, KV = 30.5 GFLOPs/MB.

Q21. [D] Tensor 6 → KV 변환에서 저장된 hidden state는 layer마다 다른가?

A. 그렇다. Layer L개라면 layer마다 별도 tensor 6 저장. 다만 본 논문의 storage-efficient tensor는 per-layer로 모두 캐시. 변환 비용은 GPU layer 1 step에 해당.

Q22. [L] Tensor 6은 LayerNorm 출력이라 다음 layer의 weight로 가야 의미가 있다. Layer L의 KV가 evict 되면 layer L의 모든 tensor 6를 다시 풀어야 하는가?

A. 그렇다. 하지만 한 번 풀면 그 layer의 attention 통과는 KV cache를 사용하므로 다른 layer 영향 없음. 변환 비용은 sequential하게 layer 단위로 누적되지만 < 100 ms 측정값 안에 들어 있다.

Q23. [L] GQA 모델에서 Mechanism 3을 비활성화하면 Bidaw 전체 ablation 결과가 어떻게 변하나?

A. Tensor caching 부분 $1.10\times$ 향상이 사라진다. 즉 누적 throughput은 $1.97\times (1.58 \times 1.25)$ 수준으로 유지. Llama-2/3, Qwen2.5+ 같은 GQA 모델에선 Mechanism 1+2만으로도 큰 효과는 유지된다는 게 본 논문이 암시하는 메시지.

Q24. [E] 그러면 GQA 모델에서 직접 평가한 결과는?

A. 평가 모델 5개 중 GQA를 명시한 모델은 없다. Qwen-7B/14B는 논문 시점에 GQA로 알려졌지만 본 논문이 사용한 weight는 MHA 변형일 가능성이 있다. 정확한 답을 위해선 논문 § 5.1의 모델 설정을 한 번 더 확인 — 발표 시 reviewer가 이걸 짚는다면 "Qwen 1.0 / 1.5 등 MHA 변형 weight 사용 가능성, 정확한 답은 논문 보충자료 필요"라고 정직하게 답변.

E. 시스템 통합과 일반화

Q25. [D] Bidaw가 vLLM, CachedAttention, FlashGen 위에 얹히는 시스템인가, 아니면 대체하는가?

A. 얹히는 쪽에 가깝다. vLLM의 PagedAttention과 continuous batching은 그대로 사용. CachedAttention/FlashGen의 inclusive caching 아이디어도 채택. 추가로 (a) dual-queue scheduler, (b) ghost-cache eviction manager, (c) history cacher를 끼워 넣는다. 직접 비교용 baseline은 CachedAttention/FlashGen이지만 구조적으로 보완 관계.

Q26. [L] Disaggregated serving (DistServe류, prefill/decode 분리)에서도 작동하는가?

A. 논문은 단일 GPU 노드. Disaggregated에서는 KV가 prefill GPU에서 decode GPU로 NVLink 또는 RDMA로 이동 — capacity layer가 SSD가 아니라 원격 GPU memory 또는 RDMA-attached pool이 된다. Bidaw의 scheduling/eviction 원칙은 그대로 적용 가능하지만 I/O 시간 $\approx$ KV size 가정은 NIC 대역 폭으로 다시 측정해야 한다. 일반화 가능하나 구체적 평가는 future work.

Q27. [L] Cross-tenant fairness — 한 큰 KV 사용자가 작은 KV 사용자들을 starvation시키는가?

A. 답변 길이 신호는 user 단위로 추적되므로 user-별 분포가 들어간다. 그러나 disk-HRRN은 모든 사용자 통합 큐에서 size-biased 우선순위. 큰 KV 사용자의 ratio가 천천히 올라가도 결국 promote는 됨. 다만 fairness QoS 보장은 명시적이지 않다 — 멀티 테넌시 전용 후속 연구가 필요.

Q28. [D] Hyperparameter 5%가 eviction trigger threshold인데 sensitivity?

A. 논문에 sensitivity analysis 없음. 너무 작으면 eviction 빈도가 ↑ (overhead ↑), 너무 크면 perf layer 활용도 ↓. 5%는 empirical하게 정한 값으로 보임. 발표 중 질문 받으면 "논문이 이 값에 대한 분석을 명시적으로 하지 않았다"고 정직하게 답할 것.

Q29. [E] Tail latency P99 -47%인데, 더 극단의 P99.9는?

A. 논문은 P99까지만 보고. P99.9 측정은 hours-scale trace가 필요할 수 있음. 본 논문 trace는 large enough하지만 P99.9 cut-off 통계 안정성은 별도 분석. **예상**: dual-queue가 promotion 시 **원래 도착 시각**을 유지하므로 P99.9 폭주는 작을 가능성 — 다만 보장은 아님.

Q30. [L] "Bidirectional awareness"라는 일반 원칙을 다른 시스템에 적용한다면?

A. 다른 **human-in-the-loop** 워크로드에 적용 가능. 예: 음성 비서 (사용자 침묵 길이 → 다음 발화 시간 추정), 게임 NPC (이전 대사 길이 → 플레이어 reaction 시간), 협업 IDE assistant (응답 길이 → 사용자 코드 작성 시간). 핵심은 **컴퓨터 출력의 길이/내용이 다음 입력의 timing을 예측하게 한다**는 단방향이 아닌 양방향 신호. 단순 throughput-bound batch serving에서는 이 신호가 약해 적용 어려움.

답변 시 자세

- 모르는 부분은 모른다고. ("이건 논문이 명시하지 않았습니다.") 추측을 단언하지 말 것.
- 한계를 미리 인정. ("이건 본 논문의 약점인데, ...")
- 숫자는 정확히. $3.58 \times / 1.83 \times / 22.4 / 0.94-0.98 / 51$ vs 30.5 — 막힘 없이 나와야 한다.